

Interface

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include "../include/estruturas.h"
```

// Função que imprime uma carta

void imprimirCarta(Carta c, int destaque) {

Lista com os símbolos → const char* naipes[] = {"\u2660", "\u2665", "\u2666", "\u2663"};

Valores → const char* vals[] = {"", "A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"};

Cores ordenadas por nome → const char* cores[] = {"\033[0m", "\033[31m", "\033[31m", "\033[0m"};

Se for de destaque imprime a cor → if (destaque) {
printf("\033[33m%s%\033[0m\t", vals[c.num], naipes[c.naipes]);
} else {
printf("%s%s%\033[0m\t", cores[c.naipes], vals[c.num],
naipes[c.naipes]);
} Se não imprime com a respectiva cor

}

// Função que percorre todas as pilhas do tabuleiro e determina qual é a maior

int obterMaxAltura(EstadoJogo *e) {

int max = 0, i = 0;

while (i < e->total_pilhas) {

if (e->pilhas[i].nome_tipo[0] == 'T' && e->pilhas[i].quantidade > max) max = e->pilhas[i].quantidade;
i++;

}

return max;

}

Retorna o valor

Se for do tipo Tabuleiro

↓

Se a quantidade for maior que o máximo, atualiza

// Função que imprime os cabeçalhos das linhas (C1, C2, C3, ...)

void imprimir_cabecalhos(EstadoJogo *e, int dica_o, int dica_d) {

int p = 0;

printf("\n=== TABULEIRO ===\n");

while (p < e->total_pilhas) {

if (e->pilhas[p].nome_tipo[0] == 'T') {

if (p == dica_o || p == dica_d) printf("\033[33mC%d\033[0m\t", p + 1);

else printf(" C%d\t", p + 1);

}

p++;

}

printf("\n");

}

Alguns imprimem cabeçalho como a pilha de tipo 'T'

Se for dica imprimem com o destaque

Se não imprimem normal

// Função que imprime as cartas que estão naquela linha do tabuleiro

void imprimir_linha_tabuleiro(EstadoJogo *e, int h, int dica_o, int dica_d)

{

int p = 0;

while (p < e->total_pilhas) {

Para pelas pilhas do tipo tabuleiro → if (e->pilhas[p].nome_tipo[0] == 'T') {

if (e->pilhas[p].quantidade > h) { Se tem cartas para chegar até h

Decide se é dica ou
mo. Abaixa a altura
carta pode ser anulada

```

    int dest = ((p == dica_o || p == dica_d) && h ==
    e->pilhas[p].quantidade - 1);
    imprimirCarta(e->pilhas[p].cartas[h], dest);
} else {
    printf("\t");
}
p++;
printf("\n");
}

```

Se não tiver altura apenas imprimir espaços vazios

// Função que imprime as outras colunas que não são do jogo.

// Se a pilha for do tipo S (Stock/ baralho), apenas aparece '?'

```

void imprimir_detalhe_pilha(Pilha *p, int num_coluna, int destaque) {
    if (destaque) printf("\033[33mC%d (%s):\033[0m ", num_coluna,
    p->nome_tipo);
    else printf("C%d (%s): ", num_coluna, p->nome_tipo);
}

```

Se a pilha tem cartas

```

if (p->quantidade > 0) {
    if (p->nome_tipo[0] == 'S') {
        printf("[ \033[34m?\033[0m ] (%d cartas ocultas)",
        p->quantidade);
    } else {
        imprimirCarta(p->cartas[p->quantidade - 1], destaque);
    }
} else {
    printf("[ Vazia ]");
}
printf("\n");
}

```

Se a pilha do Stock

imprime ?

Se não imprimir a carta (exemplo: furoes)

Se não tiver cartas

// Função que faz o ciclo por todas as pilhas que não pertencem ao tabuleiro

```

void imprimir_outras_pilhas(EstadoJogo *e, int dica_o, int dica_d) {
    int p = 0;
    printf("\n--- OUTRAS PILHAS ---\n");
    while (p < e->total_pilhas) {
        if (e->pilhas[p].nome_tipo[0] != 'T') {
            int destaque = (p == dica_o || p == dica_d);
            imprimir_detalhe_pilha(&e->pilhas[p], p + 1, destaque);
        }
        p++;
    }
    printf("=====\n");
}

```

Se não for do tabuleiro

Se não tiver destaque

// Função que imprime o tabuleiro

```

void mostrarTabuleiro(EstadoJogo *e, int dica_o, int dica_d) {
    int max_h = obterMaxAltura(e);
    int h = 0;
    imprimir_cabecalhos(e, dica_o, dica_d);
    while (h < max_h) {
        imprimir_linha_tabuleiro(e, h, dica_o, dica_d);
        h++;
    }
}

```

obtem a altura

imprime os cabeçalhos

imprime todas as linhas do tabuleiro


```
imprimir_outras_pilhas(e, dica_o, dica_d); ← Depois imprime as outras pilhas
```

```
// Função que lê os ficheiros .txt em /paciencias e imprime os nomes dos jogos disponíveis no ecrã
```

```
int listar_jogos(char ficheiros[][50]) {
```

```
DIR *d = opendir("paciencias");
```

```
struct dirent *dir; } ← Serve para olhar para um ficheiro
```

```
int count = 1;
```

```
if (d) {
```

```
while ((dir = readdir(d)) != NULL) {
```

```
if (strstr(dir->d_name, ".txt") != NULL) {
```

```
sprintf(ficheiros[count], "paciencias/%s", dir->d_name);
```

```
printf("%d. Jogar %s\n", count, dir->d_name);
```

```
count++;
```

```
}
```

```
closedir(d); ← fecha o directorio
```

```
} else {
```

```
printf("[ERRO] Pasta 'paciencias' não encontrada!\n");
```

```
}
```

```
return count;
```

```
}
```

```
// Função auxiliar para retirar instruções ao menu principal
```

```
void imprimir_cabecalho_menu(void) {
```

```
system("clear"); ← Apaga o que estava antes
```

```
printf("---- MOTOR DE PACIÊNCIAS ----\n");
```

```
}
```

```
// Função que mostra o menu de início do jogo
```

```
int menuPrincipal(char *caminho) {
```

```
char ficheiros[100][50];
```

```
int escolha;
```

```
imprimir_cabecalho_menu(); ← imprime o cabeçalho
```

```
int count = listar_jogos(ficheiros); ← imprime os jogos e recebe o nº de jogos
```

```
printf("0. Carregar Save\n1. Sair\nEscolha: "); ← imprime outras opções
```

```
scanf("%d", &escolha);
```

```
if (escolha == 0) return 1; ← Save
```

```
if (escolha > 0 && escolha < count) {
```

```
strcpy(caminho, ficheiros[escolha]);
```

```
return 2; ← Vai começar um jogo novo
```

```
}
```

```
return 0;
```

```
}
```

```
// Função que pede a jogada ao utilizador durante o jogo
```

```
void pedirJogada(int *orig, int *dest, int *qtd) {
```

```
printf("\nJogada (Origem Destino Qtd) Ex: '1 2 1': ");
```

```
scanf("%d %d %d", orig, dest, qtd);
```

```
}
```